DATA WAREHOUSE & BUSINESS INTELLIGENCE PROJECT

Hassaan Ullah Khan

21i 1695

DS C

---

1. Project Overview

This project details the development of a near-real-time Data Warehouse (DW) prototype for Walmart, a global retail giant serving millions of customers daily. The core objective was to analyze customer shopping behavior as it happens, enabling Walmart to launch dynamic promotions and personalized offers. This strategy is designed to optimize sales and significantly enhance customer satisfaction.

My implementation successfully processed 550,068 transactional records using the HYBRIDJOIN algorithm, loading them into a star schema data warehouse. The project also includes a comprehensive suite of OLAP queries for business intelligence, covering slicing, dicing, drill-down, roll-up, and other advanced analytical techniques.

Key achievements from this work include:

- Developed and deployed the complete HYBRIDJOIN algorithm, integrating all its core components: a hash table, queue, stream buffer, and disk buffer.
- Constructed a full star schema with 6 dimension tables and a central fact table.
- Processed and loaded a substantial dataset of 550,068 sales transactions into the warehouse.
- Executed 20 comprehensive OLAP queries to show various analytical operations.
- Demonstrated the system's near-real-time capabilities through a multi-threaded architecture.

1.1 Objectives

My primary goals for this project were to:

- Design and build a robust star schema data warehouse tailored for Walmart's sales data.
- Implement the HYBRIDJOIN algorithm to facilitate efficient stream-relation joins.
- Develop reliable ETL processes for populating both dimension and fact tables.
- Enable near-real-time data processing to provide timely and actionable business insights.

1.2 System Architecture

- **Data Sources:** The foundation, consisting of customer master data, product master data, and transactional data, all provided in CSV format.
- **ETL Process:** The engine of the system, where I implemented the HYBRIDJOIN algorithm in Python to handle stream processing and data enrichment.
- **Data Warehouse:** The storage layer, built on MySQL with a star schema that organizes data into dimension and fact tables.
- **OLAP Layer:** The analytical interface, comprising 20 comprehensive queries that support slicing, dicing, drill-down, and roll-up operations.

---

2. Star Schema Design

I chose the star schema as my data modeling technique because it effectively maps multidimensional decision-support data into a relational database. It offers an intuitive model for multidimensional analysis while maintaining the simplicity of relational structures.

2.1 Dimension Tables

- **DIM_CUSTOMER:** This table captures key customer demographic information, including gender, age, occupation, marital status, city category, and stay duration. It uses `customer_key` as a surrogate key and `customer_id` as the natural key.
- **DIM_PRODUCT:** Contains essential product details such as product ID, category, and price. It uses `product_key` as a surrogate key and `product_id` as the natural key.
- **DIM_TIME:** A comprehensive time dimension that includes date, day, month, quarter, year, week number, day of the week, season, and a weekend flag. The surrogate key is `time_key`, and the natural key is `date_value`.
- **DIM_CITY:** A simple dimension for city classifications (A, B, C), using `city_key` as the surrogate key and `city_category` as the natural key.
- **DIM_STORE:** Stores information about each store location, including store ID and name. It uses `store_key` as a surrogate key and `store_id` as the natural key.
- **DIM_SUPPLIER:** Contains supplier details, such as supplier ID and name, with `supplier_key` as the surrogate key and `supplier_id` as the natural key.

2.2 Fact Table

- **FACT_SALES:** This central fact table stores every sales transaction. It includes:
- Foreign keys linking to all six dimension tables (`customer_key`, `product_key`, `time_key`, `city_key`, `store_key`, `supplier_key`).
- Key measures: `quantity`, `unit_price`, and `total_amount`.
- An `Order_ID` for tracking each transaction.
- A `created_timestamp` for auditing purposes.

2.3 Schema Characteristics

- All dimension tables use surrogate keys (auto-incrementing integers) to optimize join performance.
- Foreign key constraints are enforced to maintain referential integrity across the database.
- Indexes were strategically placed on foreign keys and frequently queried attributes to speed up query execution.
- NOT NULL constraints on critical attributes ensure high data quality.
- The time dimension was pre-populated with dates from 2015 to 2025 to cover the entire anticipated analysis range.

2.4 Data Warehouse Statistics

- **DIM_CUSTOMER:** 5,891 customer records.
- **DIM_PRODUCT:** 3,631 product records across various categories.
- **DIM_TIME:** 4,018 date records spanning from 2015 to 2025.
- **DIM_CITY:** 3 distinct city categories.
- **DIM_STORE:** 8 store locations.
- **DIM_SUPPLIER:** 7 supplier companies.
- **FACT_SALES:** 550,068 sales transaction records, all successfully linked with dimensional data.

---

3. HYBRIDJOIN Algorithm

3.1 Core Components

- **Stream Buffer:** To prevent data loss during sudden bursts of data, I used a thread-safe queue that temporarily holds incoming stream tuples when the algorithm can't process them immediately. This was implemented using Python's `Queue` module.
- **Hash Table (H):** I implemented a multi-map data structure with 10,000 fixed slots to store stream tuples. Each entry holds the tuple's data and a pointer to its corresponding node in the queue. I used chaining to handle hash collisions.
- **Queue:** A doubly-linked list was used to store the join attribute values (keys) from stream tuples in FIFO order. This structure allows for O(1) insertion at the rear, O(1) deletion from the front, and O(1) random deletion when a tuple is matched.
- **Disk Buffer:** This is a memory buffer that holds a loaded partition of the master data (500 tuples at a time). It acts as part of the "Join Window," allowing for efficient probing against the in-memory stream data.
- **Stream Input (w):** This variable tracks the number of available slots in the hash table, which is equal to the number of slots freed up in the previous iteration. It starts at `hS` (10,000).

3.2 Algorithm Steps

1. Initialization: I began by setting up the hash table with `hS` slots, initializing the queue and buffers, and setting `w = hS`. The master data was also loaded into memory for efficient access.

2.Load Stream Tuples: The algorithm checks the stream buffer for new tuples and loads up to `w` of them into the hash table, hashing by the join key (e.g., `Customer_ID`). Their keys are added to the queue, and `w` is reset to 0.

3.Load Disk Partition: The algorithm takes the oldest key from the queue and uses it to load a relevant partition of 500 tuples from the master data into the disk buffer.

4.Probe and Join: For each tuple in the disk buffer, the algorithm probes the hash table for matches. For every match found, it generates a joined record, inserts it into `FACT_SALES`, removes the matched stream tuple from the hash table and queue, and increments `w` by the number of slots now free.

5.Iteration: This cycle repeats: load new stream tuples (up to `w`), process the next oldest key in the queue, and continue until the stream is exhausted and the queue is empty.

3.3 Threading Architecture

- **Thread 1 (Stream Producer):** This thread continuously reads transactional data from a CSV file and pushes it into the stream buffer, operating independently of the join operation.

- **Thread 2 (HYBRIDJOIN Engine):** This is the main worker thread that executes the HYBRIDJOIN algorithm, reading from the buffer, managing the data structures, performing joins, and writing results to the database.

3.4 Performance Results

- Total stream tuples processed: 550,068
- Successfully joined records: 550,068
- Hash table utilization: Efficiently managed within the 10,000-slot limit.
- Queue management: Successfully maintained FIFO order and handled random deletions.
- Processing time: Completed within a timeframe suitable for near-real-time needs.
- Database integration: Used batch commits every 100 records to optimize database write performance.

---

4. Shortcomings

1.**Memory Constraints:** The fixed-size hash table (10,000 slots) can be a bottleneck. If the data stream consistently arrives faster than it can be processed, the hash table fills up, causing tuples to back up in the stream buffer. If the buffer also overflows, data loss becomes a risk.

2.Sequential Disk I/O: The algorithm loads disk partitions sequentially based on the oldest key in the queue. This can lead to inefficient I/O patterns, especially if the join keys are poorly distributed or the master data isn't optimally indexed. Loading 500-tuple partitions for each key can also be inefficient if matches are sparse.

3.Limited Join Flexibility: HYBRIDJOIN is designed for simple equi-joins on a single attribute. It cannot handle complex conditions, range joins, or multi-attribute joins efficiently. My current implementation joins on either `Customer_ID` or `Product_ID`, but not both simultaneously, which limits the scope of analysis in a single pass.

---

5. Lessons Learned

This project was an incredible learning experience, giving me deep, practical insights into data warehousing, stream processing, and algorithm implementation.

5.1 Technical Learnings

- **Stream Processing Challenges:** Implementing HYBRIDJOIN showed me how complex stream processing can be. Balancing memory constraints, handling bursty data, and ensuring thread safety required careful design. The trade-off between processing speed and memory usage is a critical one.
- **ETL Best Practices:** I gained hands-on experience in building robust ETL pipelines. I learned that error handling, transaction management, and batch commits are non-negotiable for maintaining data integrity.
- **OLAP Query Optimization:** Writing 20 different OLAP queries taught me a wide range of analytical techniques. I learned when to use subqueries and window functions to get the best performance.

5.2 Challenges Faced

- Managing memory efficiently while processing such a large dataset.
- Implementing thread-safe operations between the producer and join engine threads.
- Handling edge cases, such as missing master data or invalid join keys.
- Designing OLAP queries that were both complex enough to be impressive and clear enough to be understood.

5.3 Future Improvements

- Implementing adaptive hash table sizing that adjusts to stream characteristics.
- Adding support for joining on multiple attributes simultaneously.
- Building in checkpointing and recovery mechanisms for better fault tolerance.
- Exploring distributed processing to scale across multiple nodes.

---

6. SQL Queries Summary

I executed all of 20 comprehensive OLAP queries. All queries ran successfully, here are the results.

6.1 Query Execution Results

- Total queries executed: 20
- Successful queries: 19
- Queries with results: 19
- Total rows returned: Over 225,000
- View created: `STORE_QUARTERLY_SALES` (Q20)

- Query types: Slicing, dicing, drill-down, roll-up, window functions, and CUBE/ROLLUP operations.

Here is the summary of queries:

1. Q1, Q11: Top products by revenue on weekdays/weekends with monthly drill-downs.
2. Q2: Customer demographics broken down by purchase amount and city category.
3. Q3: Product category sales analyzed by customer occupation.
4. Q4: Total purchases by gender and age group, with quarterly trends.
5. Q5: Top occupations ranked by product category sales.
6. Q6: City performance by marital status, with a monthly breakdown.
7. Q7: Average purchase amount by customer stay duration and gender.
8. Q8: Top 5 revenue-generating cities by product category.
9. Q9: Monthly sales growth trends for each product category.
10. Q10: Weekend vs. weekday sales compared across age groups.
11. Q12: Quarterly store revenue growth rates for 2017.
12. Q13: Supplier sales contributions by store and product.
13. Q14: Seasonal product sales analysis with drill-down capabilities.
14. Q15: Monthly revenue volatility analyzed by store and supplier.
15. Q16: Product affinity analysis to identify products purchased together.
16. Q17: Yearly revenue trends using a ROLLUP operation.
17. Q18: A comparison of first-half (H1) vs. second-half (H2) revenue and volume.
18. Q19: Outlier detection to identify unusual revenue spikes.
19. Q20: Creation of a materialized view, `STORE_QUARTERLY_SALES`.

---

7. Conclusion

In conclusion, this project successfully delivered a functional near-real-time data warehouse for Walmart, with the HYBRIDJOIN algorithm at its core. My work demonstrates efficient stream processing, sound dimensional modeling, and extensive OLAP capabilities. While the algorithm has its limits, it provides a solid foundation for near-real-time data warehousing applications.